

ROLE **FDE Intern**
TIME REQUIRED **2-3 hours**
FORMAT **Working prototype**
SUBMIT VIA **GitHub**

The Discount Engine Problem

A code-build challenge for candidates applying to FDE Intern, Central Tech

Background

When a customer shops on Opptra's marketplace, they see the final price at checkout — not the base price. Brands run their own discount campaigns (Rs.150 off all Natura Casa products) while platforms run their own (15% off everything on Amazon India). When both apply to the same cart item, the system must pick the one that gives the customer the maximum discount and show them a clear explanation of what offer they received.

This is a **customer-facing cart pricing engine**. The output is what a real customer reads at checkout: final price + a plain-language breakdown of which offer was applied. Always apply the rule that gives the customer the biggest saving. If rules can stack, stack them.

Your Stakeholder

C "A customer adds a Natura Casa cushion cover to their cart on Amazon India. There's a brand offer — Rs.150 off. There's also a platform offer — 15% off. The customer should see whichever gives them the better deal, shown clearly. One final price, one offer label, no surprises."

Customer cart view · Amazon India checkout

Sample Data

Active Discount Rules

RULE ID	SCOPE	APPLIES TO	TYPE	VALUE	CONDITION	STACKABLE
RULE-01	Platform	Amazon India	Percentage	15% off	—	No
RULE-02	Brand	Natura Casa	Flat	Rs.150 off	—	No
RULE-03	Platform	Flipkart	Percentage	10% off	—	Yes
RULE-04	Cart	Entire cart	Percentage	10% off	Cart total ≥ Rs.4,000	No

Cart Items

ITEM	PRODUCT	BRAND	PLATFORM	BASE PRICE	RULES THAT MATCH
ITEM-01	Cushion Cover	Natura Casa	Amazon India	Rs.1,299	RULE-01 + RULE-02 — pick max
ITEM-02	Bed Sheet Set	Natura Casa	Flipkart	Rs.849	RULE-02 + RULE-03 — stackable
ITEM-03	Wall Shelf	LivSpace Pro	Amazon India	Rs.599	RULE-01 only
ITEM-04	Ceramic Vase	LivSpace Pro	Noon	Rs.2,499	No rules match
ITEM-05	Cutting Board	Nordic Basics	Amazon India	Rs.449	RULE-01 only
ITEM-06	Desk Organiser	Nordic Basics	Flipkart	Rs.899	RULE-03 only

Discount Selection Logic

HOW TO PICK WHICH RULE TO APPLY

- When multiple non-stackable rules match the same item, **apply the one that gives the largest discount amount** in rupees. Scope does not matter — only the saving does.
- A rule marked `stackable: true` always applies **on top of** the winning non-stackable rule. If no non-stackable rule matched, a stackable rule still applies on its own.
- When no rules match an item, return the base price with a "No offers available" note.
- Cart-level rules** apply to the entire cart total after all item-level discounts are calculated. Check the condition (e.g. cart total $\geq$ Rs.4,000) against the sum of final item prices, then apply the cart discount on top of that total.

ITEM-01: RULE-01 gives $Rs.1,299 \times 15\% = Rs.194.85$ off. RULE-02 gives Rs.150 off. RULE-01 wins. Final: $Rs.1,299 \times 0.85 = \text{Rs.1,104}$

ITEM-02: RULE-02 wins (non-stackable, Rs.150 off) $\rightarrow$ Rs.699. RULE-03 is stackable $\rightarrow$ applies on top $\rightarrow Rs.699 \times 0.90 = \text{Rs.629}$

Cart offer: Item totals sum to $Rs.5,932 \geq Rs.4,000 \rightarrow$ RULE-04 applies $\rightarrow Rs.5,932 \times 0.90 = \text{Rs.5,339}$ final cart total

What You Need to Build

FOUNDATIONBuild and deploy the discount engine

—60 min

The base code is provided (see Getting Started below). It already handles CSV upload, item-level discount logic, and the results display. Run it locally, verify the expected results, then **deploy it** — Vercel, Netlify, or any platform you prefer. The live URL must be in your README. The tasks below are built on top of this deployed base.

TASK 1Cart-level offer

—30 min

Extend the engine to support cart-level discount rules. A cart rule has a minimum cart value condition — if the sum of all final item prices meets or exceeds the threshold, the percentage discount is applied to the entire cart total. The cart offer is evaluated after all item-level discounts are applied and shown as a separate discount line in the results, not per item.

The results section must clearly show: each item's final price, a cart offer row if triggered (e.g. "Cart offer: 10% off — Rs.593 saved"), and the final cart total after the cart offer is applied. If the cart total does not meet the threshold, the cart offer row should not appear.

TASK 2Natural language rule input

—50 min

Add a text field where a user can describe a new discount rule in plain English. An LLM parses it into a structured `DiscountRule` object. Show the parsed fields in a confirmation step before adding the rule — the user confirms or discards. The engine re-runs on the current cart with the new rule included.

- Input** "20% off for Natura Casa brand, stackable with other offers"
- Parsed** scope: Brand · applies to: Natura Casa · type: Percentage · value: 20% · stackable: Yes
- Input** "Rs.100 flat discount on all Flipkart items"
- Parsed** scope: Platform · applies to: Flipkart · type: Flat · value: Rs.100 · stackable: No
- Input** "10% off if cart value is more than Rs.5,000"
- Parsed** scope: Cart · type: Percentage · value: 10% · min cart value: Rs.5,000
- Ambiguous** "Give a discount for big orders" — no value, no threshold. Surface as unresolvable, ask the user to be more specific.

TASK 3PDF cart upload

—40 min

Add a PDF upload option to the cart input. When a user uploads a cart PDF, extract the items, convert them to `CartItem` objects, and replace the current cart. The engine re-runs automatically with the existing active rules. The expected PDF structure is a simple table with columns: **Product, Brand, Platform, Base Price.**

Order #OP-9921 Date: 15 Jan 2025			
Product	Brand	Platform	Base Price
Cushion Cover	Natura Casa	Amazon India	Rs.1,299
Bed Sheet Set	Natura Casa	Flipkart	Rs.849
Wall Shelf	LivSpace Pro	Amazon India	Rs.599
Ceramic Vase	LivSpace Pro	Noon	Rs.2,499
Cutting Board	Nordic Basics	Amazon India	Rs.449
Desk Organiser	Nordic Basics	Flipkart	Rs.899

Expected Results

Verify the base engine produces these before building the tasks.

ITEM	BASE PRICE	RULE(S) APPLIED	FINAL PRICE	STATUS
ITEM-01	Rs.1,299	RULE-01 wins (Rs.195 saving > Rs.150)	Rs.1,104	Max discount
ITEM-02	Rs.849	RULE-02 (–Rs.150) + RULE-03 stacked (–10%)	Rs.629	Stacked
ITEM-03	Rs.599	RULE-01 (15% off)	Rs.509	Discount applied
ITEM-04	Rs.2,499	No rules match	Rs.2,499	No offer
ITEM-05	Rs.449	RULE-01 (15% off)	Rs.382	Discount applied
ITEM-06	Rs.899	RULE-03 (10% off, stackable)	Rs.809	Discount applied
Cart Total before offer		Rs.5,932		
Cart Offer — RULE-04		Rs.5,932 ≥ Rs.4,000 → 10% off entire cart → –Rs.593	–Rs.593	Cart offer
Final Cart Total		Rs.5,339		

Getting Started

Step 1 — Fork the base repo

The base code is available at: <https://github.com/opptra/discount-engine-assignment>

Click **Fork** (top right) → Create fork. This creates your own copy to work from. Submit your fork's URL, not the original.

Step 2 — Run locally

```
git clone https://github.com/YOUR_USERNAME/discount-engine
cd discount-engine
npm install
npm run dev
```

Open <http://localhost:5173>. You'll see two upload areas — one for rules, one for cart.

Step 3 — Upload the sample CSVs and verify

Both files are in `sample-data/` inside the repo. Upload them and click **Calculate Discounts**. Confirm your output matches the expected results table above before starting the tasks.

rules.csv

```
rule_id,scope,applies_to,type,value,stackable,min_cart_value
RULE-01,platform,Amazon India,percentage,15,false,
RULE-02,brand,Natura Casa,flat,150,false,
RULE-03,platform,Flipkart,percentage,10,true,
RULE-04,card,,percentage,10,false,4000
```

cart.csv

```
item_id,product,brand,platform,base_price
ITEM-01,Cushion Cover,Natura Casa,Amazon India,1299
ITEM-02,Bed Sheet Set,Natura Casa,Flipkart,849
ITEM-03,Wall Shelf,LivSpace Pro,Amazon India,599
ITEM-04,Ceramic Vase,LivSpace Pro,Noon,2499
ITEM-05,Cutting Board,Nordic Basics,Amazon India,449
ITEM-06,Desk Organiser,Nordic Basics,Flipkart,899
```

Step 4 — Deploy before submitting

Run `npm run build` and deploy the `dist/` folder to Vercel, Netlify, or any static host. Add the live URL to your README. The Loom walkthrough must be recorded on the live deployment — not on localhost.

How We'll Evaluate This

CORRECTNESS Does the base engine produce the right results? Does Task 1 correctly parse and apply a new rule? Does Task 2 replace the cart and re-run?
LLM INTEGRATION QUALITY Is the parsed rule validated before use? Is the confirmation step clear? Does ambiguous input surface a useful error rather than crashing?
CODE CLARITY Are the two new input paths clearly separated from the engine logic? Can someone add a third input mode without touching the discount calculator?

CUSTOMER-FACING OUTPUT

Are reasoning strings accurate after a new rule is added or a new cart is loaded? Does ITEM-04 produce a meaningful "no offer" message?

TRADEOFF JUDGMENT

How do they handle a malformed PDF row? What happens when the LLM returns an invalid rule? What happens when cart total is just below the threshold? Did they make deliberate decisions or just hope for the happy path?

What Not to Build

- No database or file persistence — in-memory state for the session is sufficient.
- A backend is optional — both tasks can be done client-side. Only add a server if PDF parsing requires it.
- Do not rebuild the engine to accommodate the new inputs — the inputs should adapt to the engine, not the other way around.

What to Submit

REQUIRED

GitHub repo link

Your fork of <https://github.com/opptraltd/discount-engine-assignment> — public or accessible. README must include the live deployment URL and explain how to run locally in under 3 steps.

Ground Rules

- Use any language, framework, and AI coding tools — Cursor, Claude Code, Copilot. We evaluate judgment over the output.
- If you disagree with a design choice — the confirmation step, the PDF format assumption — document it and propose an alternative. Good reasoning for a different call beats silent compliance.